Cheat Sheet: Writing Effective Tools for AI Agents

Source(s): https://www.anthropic.com/engineering/writing-tools-for-agents | react-agent project (W02D1–D4)

Date: 2026-06-07

---

1. Big Picture

• Tools are a **contract between deterministic systems and non-deterministic agents** — unlike APIs (which always behave identically), agents can call tools in unexpected sequences, skip them, or hallucinate them.

• **More tools ≠ better outcomes.** Each tool added competes for context and can dilute agent focus. Fewer, well-designed tools consistently outperform large toolsets.

• Tool descriptions are **prompt engineering** — they are injected into the agent's context and directly steer which tool gets called, when, and with what arguments.

• The ReAct pattern (Thought → Action → Observation) makes tool use transparent and auditable. `stop_sequences=["Observation:"]` is the critical guard that prevents hallucinated observations.

• The best tools are ergonomic for agents AND intuitive for humans — these goals align, not conflict.

---

2. Mental Model

Think of tools as **onboarding docs for a new hire**, not API docs for a developer. A developer reads the type signature and figures out the rest. A new hire needs context: *what is this for, when do I use it, what do I do if it fails, what should I never do with it?* Write every tool description with that level of explicitness.

An agent with bad tools is like a new hire given a 300-page reference manual with no index — technically everything is there, but they'll still call the wrong person for the wrong thing.

---

3. Key Concepts

Concept	Simple meaning	Remember it as
Tool contract	Agreement between code (deterministic) and agent (non-deterministic)	API for a human, not a machine
Namespacing	Grouping tools under common prefixes (`asana_search`, `slack_search`)	File folders for tools
Agent affordances	What actions the agent perceives as available to it	The agent's "menu"
Token efficiency	Returning only high-signal data, avoiding raw dumps	Quality > quantity in context

Concept	Simple meaning	Remember it as
Agent-facing description	Docstring written to guide the agent's decision-making, not the developer's	`WHEN to use / NOT to use / ON FAILURE do`
FM-REASONING	Failure mode where agent reasons incorrectly because it lacks the right tool (e.g. counting without `run_python`)	The gap between capability and tooling
Actionable errors	Error messages that tell the agent exactly what to fix and how	Stack trace → recipe
Stall detection	Detecting repeated identical tool calls and triggering reflection	Infinite loop guard
Triggered reflection	Self-critique injected only on error or stall, not every N steps	Airbag, not seatbelt

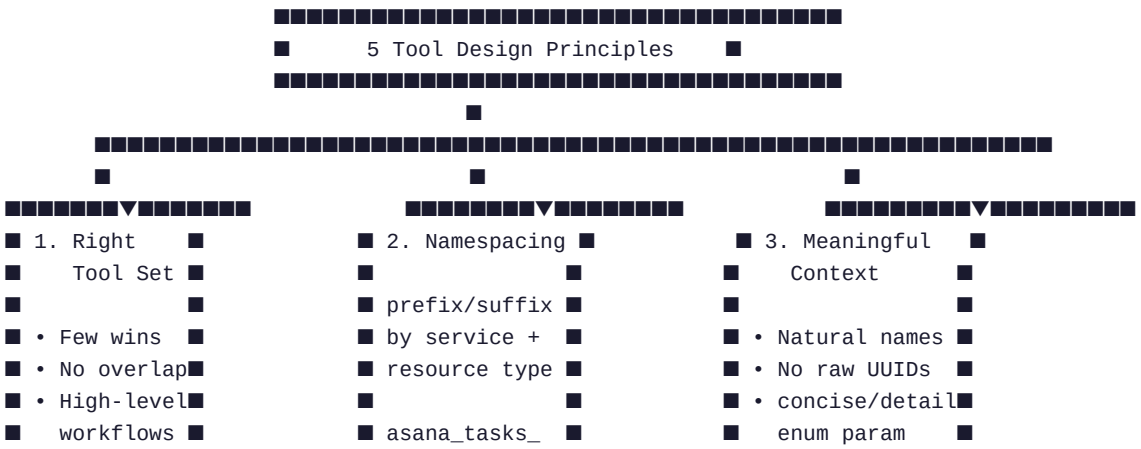
---

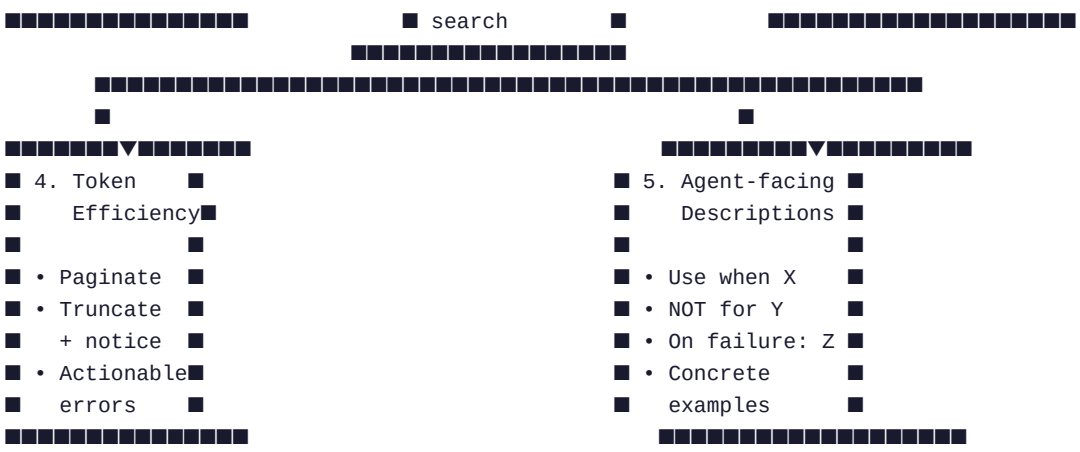
4. Chronological Steps — Building & Improving Tools

1. **Build a prototype** — wrap tools in a local MCP server, connect to Claude Code or Claude Desktop, test manually.
2. **Generate eval tasks** — grounded in real workflows, not toy examples. Tasks should require multiple tool calls.
3. **Define verifiers** — `pass_if` lambda per task. Match broad patterns (case-insensitive, synonyms), not exact strings.
4. **Run baseline** — measure pass rate, avg steps, token count, tool error rate.
5. **Analyze failures** — classify into failure modes (FM-LOOP, FM-TOOL-ERR, FM-HALLUC, FM-SEARCH, FM-REASONING). Read agent CoT + raw transcripts.
6. **Refactor tools** — apply the 5 principles below. One change at a time if possible.
7. **Re-run eval on held-out set** — confirm improvement isn't overfitting to training tasks.
8. **Repeat** — this is iterative. Claude Code can analyze transcripts and refactor tools automatically.

---

5. Diagram — Tool Design Principles





ReAct Loop (with stop_sequences guard):

```
User question
     ■
     ▼
[LLM] Thought: ...
      Action: tool(arg)     ← LLM stops here (stop_sequences=["Observation:"])
     ■
     ▼
[Code] execute_tool(name, arg) → real result
     ■
     ▼
[Conversation] "Observation: <result>"
     ■
     ▼
[LLM] Thought: ...  →  Answer: ...  (or loop again)
```

---

6. Critical Info

• `stop_sequences=["Observation:"]` **is non-negotiable.** Without it, the model writes its own fake tool results. This is the single most important implementation detail in any ReAct agent.

• **Removing a redundant tool is a win.** `calc` + `run_python` overlap caused agent hesitation in 2/20 eval tasks. Removing `calc` resolved it. Fewer choices = better decisions.

• **Binary pass rate hides quality.** C1 (FastMCP search): naive agent hallucinated a confident answer, reflective agent honestly said "unable to find." Both "passed." Track qualitative answer quality separately.

• **Eval task design is as hard as agent design.** 2 of our 3 failures in W02D2 were eval bugs (wrong expected answer, too-strict pass criterion) — not agent bugs. Fix evals first before blaming the agent.

• **Fixed reflection wastes tokens on easy tasks.** In our 60-run A/B, triggered reflection (error/stall only) matched fixed reflection (every N steps) at 20/20 with no overhead on smooth runs.

• **Prefix vs suffix namespacing matters.** The article notes non-trivial eval performance differences between `service_resource_action` vs `action_resource_service`. Always measure, never assume.

• **Actionable errors prevent wasted steps.** "file not found" → agent retries blindly. "file not found — files in this dir: ..." → agent self-corrects in one step.

• **Don't** `list_all`**, build** `search_targeted`**.** Returning all contacts / all logs wastes context. An agent reading 10,000 tokens of irrelevant data is an agent burning steps and money.

---

7. Mini Example — Before / After Tool Refactor

Before (developer-facing, v1):

```
def search(query: str) -> str:
    """
    Search for information using DuckDuckGo Instant Answer API (no key needed).
    Falls back to a clear "no result" message if the API returns nothing useful.
    """
    ...
    return f"No direct result found for '{query}'. Try rephrasing."
```

After (agent-facing, v2):

```
def web_search(query: str) -> str:
    """
    Search the web for factual information about a topic.

    Use this when you need external knowledge not available in local files.
    Do NOT call web_search for math — use run_python instead.
    Do NOT call web_search for local file contents — use file_read instead.

    If the search returns no result, try:
      1. A shorter, simpler query
      2. A different angle on the same topic
      3. Answering from your own knowledge if you are confident
    """
    ...
    return (
        f"No result found for '{query}'.\n"
        "Suggestions:\n"
        "  - Try a shorter or rephrased query\n"
        "  - Use file_read if the information is in a local file\n"
        "  - Answer from your own knowledge if you are confident"
    )
```

Result: Agent now knows exactly when to use it, when not to, and what to do when it fails — no wasted steps.

---

8. References

- Writing effective tools for agents — Anthropic Engineering — 5 principles, eval methodology, MCP tool design
- Yao et al. ReAct, ICLR 2023 — Thought/Action/Observation pattern, stop_sequences rationale
- Shinn et al. Reflexion, 2023 — Self-critique loop theory behind reflective agent
- `tools/v1.py` vs `tools/v2.py` — live before/after comparison in this repo
- `docs/tool_design_delta.md` — detailed table of all 5 principles applied to this project
- `evals/failure_taxonomy.md` — empirical failure mode analysis (20-task baseline)
- `evals/ab_comparison.md` — reflection A/B results (naive vs fixed vs triggered)